Deliverable 3: SQL Operations, Multi-Table Joins & Complex Queries

Project: Intelligence Fusion Platform • **Course:** DBMS Mini Project (UE24CS395)
Authors: Saatvik Gupta (PES1UG24CS395) & Satyam Goyal (PES1UG24CS424) • PES University (2026–2027)

1. DML CRUD Workflows & Transaction Savepoints

Demonstrated in database/sql/dml-examples.sql, all state mutations execute inside transactional boundaries with explicit rollback guarantees to protect evidentiary admissibility:

```
-- Transactional Case Creation with Entity Association & SAVEPOINT
BEGIN;
INSERT INTO cases (case_id, title, description, case_type, status, priority, lead_officer_id)
VALUES ('CASE-2026-099', 'Decoy Freight Operation', 'Intercept operation for stolen electronics.', 'Smuggling', 'Open', 'High', 'OFF

SAVEPOINT after_case_insert;

INSERT INTO case_people (case_id, person_id, role_in_case)
VALUES ('CASE-2026-099', 'PER-001', 'Primary Target');

-- Conditional Rollback demonstration
-- ROLLBACK TO SAVEPOINT after_case_insert;
COMMIT;
```

2. Multi-Table Complex Joins & Correlated Subqueries

```
-- Query 1: 4-Way Multi-Table Relational Join
-- Associates suspects with vehicles and seized digital evidence under lead investigator
SELECT
    c.case_id, c.title AS case_title,
    p.full_name AS suspect_name, cp.role_in_case,
    v.make || ' ' || v.model AS vehicle_details, v.license_plate,
    e.title AS seized_evidence, e.storage_reference,
    o.full_name AS lead_investigator
FROM cases c
JOIN case_people cp ON c.case_id = cp.case_id
JOIN people p ON cp.person_id = p.person_id
LEFT JOIN case_vehicles cv ON c.case_id = cv.case_id
LEFT JOIN vehicles v ON cv.vehicle_id = v.vehicle_id
LEFT JOIN evidence e ON c.case_id = e.case_id
JOIN officers o ON c.lead_officer_id = o.officer_id
WHERE c.status IN ('Open', 'Under Investigation')
ORDER BY c.created_at DESC;

-- Query 2: Correlated Subquery
-- Identifies cases with evidence item counts strictly above department average
SELECT c.case_id, c.title, c.priority,
       (SELECT COUNT(*) FROM evidence e WHERE e.case_id = c.case_id) AS total_evidence
FROM cases c
WHERE (SELECT COUNT(*) FROM evidence e WHERE e.case_id = c.case_id) >
      (SELECT AVG(ev_count) FROM (
          SELECT COUNT(*) AS ev_count FROM evidence GROUP BY case_id
      ) sub)
ORDER BY total_evidence DESC;
```